

AWS Hosting Requirements

AI Voice Agent (Exotel Telephony) — sized for 20 simultaneous calls

1. Confirmed Stack

Reviewed against the project codebase (backend/app/providers, requirements.txt, config.py). The pipeline is fully API-based for every AI-heavy stage — STT, TTS, LLM and embeddings all call hosted providers. The only on-server compute is lightweight CPU signal processing. **No GPU is required anywhere in this stack.**

Component	Implementation	Runs where
Speech-to-text (STT)	Sarvam Saarika API	Cloud API (Sarvam)
Text-to-speech (TTS)	Sarvam Bulbul API	Cloud API (Sarvam)
LLM (conversation)	Google Gemini 2.5 Flash API	Cloud API (Google)
Embeddings	Gemini embedding-001 API	Cloud API (Google)
Voice activity detection	Silero VAD (ONNX, CPU)	Local — onnxruntime (CPU build)
Noise gate / AGC / AEC	Pure NumPy DSP	Local — CPU, sub-ms per frame
Speaker verification	MFCC + cosine similarity (NumPy)	Local — CPU, disabled by default
Knowledge base / RAG	Qdrant (optional) or in-memory fallback	Local (in-memory) or small container
Call data (mock tools)	SQLite	Local file — move to Postgres for real production
Telephony gateway	Exotel bidirectional Voicebot WebSocket	Already implemented in backend/app/telephony/exotel.py

requirements.txt confirms this: onnxruntime (CPU build, not onnxruntime-gpu), no torch, no transformers, no CUDA dependency anywhere.

2. Compute Sizing — 20 Concurrent Calls

Workload is I/O-bound (waiting on Sarvam/Gemini responses) with light continuous CPU work (VAD + AGC + AEC + noise-gate running per audio frame, for every live call). This does not require a GPU instance at any point.

Resource	Recommendation	Why
App server	2 x c6i.large (2 vCPU / 4 GB) minimum, or c6i.xlarge (4 vCPU / 8 GB) for headroom	Two instances (not one) so a crashed instance doesn't drop all 20 live calls
Deployment target	EC2 Auto Scaling Group (min 2, scale on CPU / connection count) or ECS Fargate	Horizontal scale-out beyond the pilot
Load balancer	ALB or NLB with WebSocket support, sticky sessions	/ws/call and /ws/exotel are long-lived WebSocket connections
Session store	ElastiCache Redis (cache.t4g.micro)	Replace in-memory session dict once running >1 instance (see project's own PRODUCTION_CHECKLIST.md)

Vector DB (optional)	Skip for pilot — in-memory fallback ships in code. Add Qdrant (small container) only once KB grows.	KB is currently a handful of YAML files
Database	SQLite is fine for a 20-call pilot; migrate to RDS Postgres before real scale	Already flagged in the project checklist
TLS / domain	ACM certificate + Route 53 domain	Exotel requires a wss:// (TLS) endpoint
GPU instance	Not required	All AI inference is via external APIs

3. AWS Services Checklist

- VPC with public subnets (+ private subnet if adding RDS/Redis)
- EC2 (or ECS Fargate) running the FastAPI/uvicorn app
- Application Load Balancer or Network Load Balancer (WebSocket-capable, sticky sessions)
- ACM — TLS certificate for the wss:// endpoint
- Route 53 — domain for the app (e.g. voice.yourdomain.com)
- Auto Scaling Group — min 2 instances across 2 Availability Zones
- ElastiCache for Redis — shared WebSocket/session state across instances
- RDS Postgres — production data store (replaces SQLite; not urgent for pilot)
- Security Group — inbound 443 restricted to Exotel's published outbound IP ranges
- CloudWatch — CPU / memory / active-connection alarms, log aggregation
- Secrets Manager (or SSM Parameter Store) — SARVAM_API_KEY, GEMINI_API_KEY, EXOTEL_API_KEY/TOKEN

4. Deployment Steps

- 1. Provision VPC, subnets, security groups.
- 2. Launch 2 x EC2 instances (or ECS Fargate service) in an Auto Scaling Group across 2 AZs.
- 3. Deploy the FastAPI app; set SARVAM_API_KEY, GEMINI_API_KEY, EXOTEL_API_KEY, EXOTEL_API_TOKEN via Secrets Manager / environment.
- 4. Put the app behind an ALB/NLB with a WebSocket-capable target group and sticky sessions.
- 5. Attach an ACM certificate; point a Route 53 domain at the load balancer.
- 6. Restrict inbound 443 to Exotel's published outbound IP ranges (plus your own for testing).
- 7. Stand up ElastiCache Redis and wire it in for shared session state (needed once running >1 instance).
- 8. In Exotel's App Bazaar, configure the Voicebot Applet to point at wss://your-domain/ws/exotel?sample-rate=16000.
- 9. Load-test at 20+ concurrent calls before go-live to confirm CPU sizing holds (the project's own checklist targets a 100-call load test).
- 10. Set up CloudWatch alarms and confirm auto-restart / health checks on /health.

Prepared from a review of the project codebase (ARCHITECTURE.md, requirements.txt, config.py, backend/app/providers, backend/app/telephony/exotel.py, docs/PRODUCTION_CHECKLIST.md) plus current AWS and Exotel public pricing pages. Cost figures are estimates — verify before budgeting.